

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY
COURSE CODE: CSA5112

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Debugging & Optimisation Task : 10%

QUESTIONS:

Total Marks: 50

Annexure A

Code of Conduct:

I C SANGITA(192373059) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

csangita

Annexure A

Debugging & Optimisation Task

1. Debugging Hash Function Implementation (SHA Family)

A Python program implementing a SHA-based hashing algorithm produces inconsistent hash outputs for the same input.

Identify errors in message padding and block processing steps. Fix issues related to bitwise operations and endianness handling.

Optimize the implementation to improve processing speed for large files (≥ 50 MB). Evaluate how secure hash functions improve integrity compared to basic checksum methods.

2. Optimization of Secure Socket Layer (SSL/TLS) Simulation

A Python-based simulation of SSL/TLS handshake shows delays and occasional handshake failures.

Debug issues in certificate validation and key exchange steps. Optimize the handshake process to reduce latency.

Refactor the code to reuse session keys efficiently (session resumption). Analyze the performance vs security trade-offs in TLS optimization.

3. Debugging and Optimizing Digital Signature Algorithm (DSA)

A Python implementation of DSA generates invalid signatures during verification.

Identify issues in parameter selection and random number generation. Fix errors in signature generation and verification steps.

Optimize modular arithmetic operations for faster execution. Evaluate the impact of randomness quality on signature security.

4. Debugging Key Management System

A Python-based key management system fails to securely store and retrieve cryptographic keys.

Identify flaws in key storage (plaintext storage, improper encryption).

Fix issues related to key lifecycle management (generation, distribution, rotation). Optimize secure storage using encryption and access control mechanisms.

Analyze how poor key management can compromise an entire cryptographic system.

5. Optimization of Secure File Transfer Protocol (SFTP-like System)

A Python script simulating secure file transfer experiences slow transfer speeds and occasional data corruption.

Debug issues in encryption/decryption synchronization. Fix errors in packet handling and integrity verification. Optimize throughput for large file transfers (≥ 100 MB).

Compare the optimized system with insecure file transfer methods and justify the improvements.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

		documentatio n			
--	--	----------------	--	--	--

ANSWERS:

Question 1: Debugging hash function implementation (SHA family)

Aim

To identify and fix errors in a SHA-based hashing implementation, optimize it for hashing large files (≥ 50 MB), and evaluate why SHA provides better integrity protection than basic checksum methods.

Problem statement

A Python implementation of a SHA-based hashing algorithm produces different hash values for the same input. This indicates errors in the message padding, block processing, bitwise operations, or endianness handling.

Errors identified

1. Incorrect message padding

SHA algorithms require the message length to be padded correctly.

Common mistakes:

- Missing the 1 bit after the message
- Incorrect number of zero bits
- Appending message length in bytes instead of bits
- Wrong total block size (512 bits for SHA-256)

2. Incorrect block processing

SHA-256 processes data in 512-bit (64-byte) blocks.

Common errors:

- Reading blocks of incorrect size
- Skipping the last block
- Incorrect message schedule generation

3. Bitwise operation errors

SHA uses:

- Right rotations
- Right shifts
- XOR
- AND
- NOT

Using normal shifts instead of rotations produces incorrect hashes.

4. Endianness handling

SHA-256 uses big-endian byte order.

Incorrect use of little-endian conversion changes word values and results in inconsistent hashes.

Debugging the implementation

Incorrect code

```
length = len(message)
message += b'\x80'
message += b'\x00' * (64 - len(message) % 64)
message += length.to_bytes(8, 'little')
```

Problems:

- Length stored in bytes
- Little-endian used
- Padding calculation incorrect

Correct code

```
length = len(message) * 8
```

```
message += b'\x80'
```

```
while (len(message) % 64) != 56:
```

```
    message += b'\x00'
```

```
message += length.to_bytes(8, 'big')
```

This correctly appends:

- 1 bit
- Required zero bits
- 64-bit message length in big-endian

Correct bitwise rotation

Incorrect rotation

```
def rotr(x, n):
```

```
    return x >> n
```

Correct rotation

```
def rotr(x, n):
```

```
    return ((x >> n) | (x << (32 - n))) & 0xffffffff
```

This ensures proper 32-bit circular rotation.

Correct block processing

Each 64-byte block must be converted into sixteen 32-bit words.

```
import struct
```

```
words = struct.unpack('>16I', block)
```

The > symbol specifies big-endian format.

Optimized implementation for large files

For files larger than 50 MB, loading the entire file into memory is inefficient.

Optimized approach

```
import hashlib
```

```
def sha256_file(filename):
```

```
    sha = hashlib.sha256()
```

```
    with open(filename, 'rb') as f:
```

```
        while True:
```

```
            chunk = f.read(1024 * 1024)
```

```
            if not chunk:
```

```
                break
```

```
            sha.update(chunk)
```

```
    return sha.hexdigest()
```

Advantages

- Low memory usage
- Faster processing
- Suitable for very large files
- Uses optimized C implementation in hashlib

Optimization technique

Technique	Benefit
Chunk-based reading	Reduces memory consumption
Use hashlib	Faster native implementation
1 MB buffer	Reduces I/O overhead
Avoid string conversions	Improves execution speed
Correct endianness	Ensures consistent hashes

Performance analysis:

For a 100 MB file:

Method	Approximate Time
--------	------------------

Naive implementation	4–6 s
----------------------	-------

Chunk processing	2–3 s
------------------	-------

hashlib optimized	0.5–1.0 s
-------------------	-----------

The optimized version provides significantly better throughput.

Secure hash vs checksum

Checksum

A checksum simply adds or combines bytes.

Example:

- File A checksum = 1050
- File B checksum = 1050

Different files may produce the same checksum.

SHA-256

SHA-256 generates a 256-bit cryptographic digest.

Properties:

- Collision resistant
- Preimage resistant
- Avalanche effect
- Tamper detection

Changing even one bit completely changes the hash value.

Example:

- Hello → 185f8db3...
- hello → 2cf24dba...

The outputs are entirely different.

Integrity comparison

Feature	Checksum	SHA-256
Detects accidental errors	Yes	Yes
Detects malicious modification	No	Yes
Collision resistance	Poor	Strong
Cryptographic security	No	Yes
Suitable for digital signatures	No	Yes

Conclusion

The inconsistent SHA output was caused by incorrect message padding, block processing, bitwise rotations, and endianness handling. After correcting these issues and implementing chunk-based processing with hashlib, the algorithm produces consistent hashes and performs efficiently for files larger than 50 MB. Compared to basic checksum methods, SHA-256 provides strong integrity protection, collision resistance, and security against intentional data tampering, making it suitable for secure file verification and cryptographic applications.

Question 2: Optimization of secure socket layer (SSL/TLS) simulation

Aim

To debug certificate validation and key exchange errors in an SSL/TLS handshake simulation, optimize the handshake process, and analyze the performance and security trade-offs.

Problem statement

A Python-based SSL/TLS handshake simulation experiences slow connection establishment and occasional handshake failures. The issues occur during certificate validation and session key exchange.

Errors identified

Certificate validation errors

Common problems:

- Expired certificates
- Invalid certificate signatures
- Incorrect hostname verification
- Missing trusted Certificate Authority (CA)

These errors cause the handshake process to fail.

Key exchange errors

The client and server may generate different shared keys due to incorrect key exchange implementation, resulting in handshake failure.

Corrected code

```
import ssl
context = ssl.create_default_context()
context.check_hostname = True
context.verify_mode = ssl.CERT_REQUIRED
```

This ensures proper certificate verification and secure communication.

Handshake optimization

Session resumption

TLS can reuse previously established session keys.

```
session_cache = {}
if server in session_cache:
    session_key = session_cache[server]
```

This avoids repeating the complete handshake.

Optimization techniques

- Enable session resumption.
- Reuse cached session keys.
- Reduce unnecessary certificate verification.
- Use efficient cryptographic libraries.

These methods reduce connection setup time.

Performance improvement

Method	Handshake Time
Full TLS handshake	200–300 ms
Session resumption	20–50 ms

Session resumption significantly reduces latency.

Security vs performance

Performance benefits

- Faster connections
- Lower CPU usage
- Reduced network overhead
- Better performance for repeated client connections

Security considerations

- Long-term reuse of session keys increases risk if a key is compromised.
- Session keys should expire after a limited period.
- Certificate validation must never be disabled.

A balanced approach is to use **short-lived session keys** with proper certificate verification.

Conclusion

The SSL/TLS handshake failures were caused by certificate validation and key exchange issues. After enabling proper certificate verification and implementing **session resumption**, the handshake process became faster and more reliable. TLS optimization improves performance, but security must be maintained through certificate validation and periodic session key rotation.

Question 3: Debugging and optimizing digital signature algorithm (DSA)

Aim

To identify and fix errors in a Python implementation of the Digital Signature Algorithm (DSA), optimize modular arithmetic operations, and evaluate the importance of randomness in digital signature security.

Problem statement

A Python implementation of DSA generates invalid signatures during verification. The errors occur due to incorrect parameter selection, weak random number generation, and mistakes in the signature generation and verification process.

Errors identified

Incorrect parameter selection

DSA requires valid parameters p , q , and g . If these values are not generated correctly, signature verification fails.

Weak random number generation

The private random value k must be unique and unpredictable for every signature. Reusing or predicting k can reveal the private key.

Signature verification errors

Verification fails if:

- Incorrect modular arithmetic is used
- The wrong public key is applied
- Hash values are calculated incorrectly

Corrected code

```
import secrets
```

```
k = secrets.randbelow(q - 1) + 1
```

This generates a secure random value for each signature.

Signature verification

```
valid = (v == r)
```

The signature is accepted only when the computed value v matches the received value r .

Optimization techniques

Faster modular arithmetic

Use Python's built-in modular exponentiation.

```
pow(base, exponent, modulus)
```

This is much faster than manual exponentiation.

Efficient hash computation

Use SHA-256 through the hashlib library instead of custom hashing code.

Reuse validated parameters

Generate DSA parameters once and reuse them for multiple signatures.

Performance improvement

Method	Performance
Manual exponentiation	Slow
<code>pow()</code> with modulus	Fast
Optimized hashing	Faster verification

The optimized implementation reduces signature generation and verification time.

Impact of randomness on security

The quality of randomness directly affects DSA security.

Weak randomness may cause:

- Private key exposure
- Signature forgery
- Replay attacks
- Complete compromise of the signing system

Using a secure random number generator ensures that each signature uses a fresh and unpredictable value of k .

Conclusion

The invalid signatures were caused by incorrect parameter selection, weak random number generation, and verification errors. Using secure randomness, correct modular arithmetic, and Python's optimized `pow()` function improves both correctness and performance. High-quality randomness is essential because poor randomness can completely break the security of the Digital Signature Algorithm.

Question 4: Debugging key management system

Aim

To identify and fix security flaws in a Python-based key management system, optimize secure key storage, and analyze the impact of poor key management on cryptographic security.

Problem statement

A key management system stores cryptographic keys insecurely and fails to manage the key lifecycle properly. This can lead to unauthorized access and compromise of encrypted data.

Errors identified

Plaintext key storage

Keys are stored directly in a file.

with `open('key.txt', 'w')` as `f`:

```
f.write(secret_key)
```

Anyone with file access can read the key.

Improper key lifecycle management

Common issues:

- Keys are never rotated.
- Expired keys remain active.
- Old keys are not removed securely.
- No access control is implemented.

Corrected code

Keys should be encrypted before storage.

```
from cryptography.fernet import Fernet
```

```
cipher = Fernet(master_key)
```

```
encrypted_key = cipher.encrypt(secret_key.encode())
```

This protects the key even if the storage file is accessed.

Key lifecycle management

A secure system should support:

- **Key generation** using a secure random generator.
- **Key distribution** through encrypted channels.
- **Key rotation** after a fixed time period.
- **Key revocation** when a key is compromised.
- **Secure deletion** of old keys.

Access control optimization

Restrict key access to authorized users only.

- User authentication
- Role-based permissions
- Audit logs
- Limited administrator access

These controls reduce the risk of key theft.

Optimization techniques

Technique	Benefit
Encrypted key storage	Protects stored keys
Automatic key rotation	Reduces exposure time
Access control	Prevents unauthorized use
Secure backup	Protects against key loss

Impact of poor key management

If keys are compromised:

- Encrypted files can be decrypted.
- Digital signatures can be forged.
- Secure communication can be intercepted.
- Authentication systems can be bypassed.

Even a strong encryption algorithm becomes ineffective if the encryption key is stolen.

Conclusion

The key management system was vulnerable because it stored keys in plaintext and lacked proper lifecycle management. Encrypting stored keys, implementing **key rotation**, and enforcing **access control** significantly improve security. Effective key management is essential because the security of a cryptographic system depends primarily on the protection of its secret keys.

Question 5: Optimization of secure file transfer protocol (SFTP-like system)

Aim

To debug encryption and packet handling errors in a secure file transfer system, optimize transfer speed for files larger than 100 MB, and compare the optimized system with insecure file transfer methods.

Problem statement

A Python-based secure file transfer system experiences slow transfer speeds and occasional data corruption during file transmission. The problems are caused by encryption/decryption synchronization errors, incorrect packet handling, and weak integrity verification.

Errors identified

Encryption and decryption synchronization

Data corruption occurs when the sender and receiver use different encryption states or keys.

Packet handling errors

Common problems:

- Incorrect packet order

- Missing packets
- Duplicate packets
- Incomplete packet reads

These errors can damage the transferred file.

Weak integrity verification

If packets are accepted without integrity checking, corrupted or modified data may be written to the file.

Corrected code

A hash can be used to verify data integrity.

```
import hashlib
```

```
digest = hashlib.sha256(data).hexdigest()
```

The receiver computes the same hash and compares it with the sender's hash.

Throughput optimization

Large files should be transferred in chunks.

```
CHUNK = 1024 * 1024
```

```
while chunk := file.read(CHUNK):
```

```
    send(chunk)
```

Using **1 MB chunks** reduces memory usage and improves transfer speed.

Optimization techniques

- Transfer files in fixed-size chunks.
- Use buffered I/O.
- Verify packet integrity with SHA-256.
- Reuse secure sessions when possible.
- Reduce unnecessary encryption initialization.

Performance improvement

Method	Transfer Speed
Small packets	Slow
1 MB chunk transfer	Fast
Buffered transfer	Faster

Chunk-based transfer provides better throughput for files larger than 100 MB.

Comparison with insecure file transfer

Feature	Insecure Transfer	Optimized Secure Transfer
Encryption	No	Yes
Integrity verification	No	SHA-256
Authentication	No	Yes
Data corruption detection	No	Yes
Large file support	Limited	Efficient

The optimized secure system protects both **confidentiality and integrity**.

Conclusion

The file transfer system was improved by fixing encryption synchronization, packet handling, and integrity verification. Using SHA-256 hashing, chunk-based transfer, and buffered I/O significantly increased throughput and prevented data corruption. Compared with insecure file transfer methods, the optimized SFTP-like system provides secure, reliable, and efficient transmission of large files.